

Minitutorial de las cartas de YuGiOh en Java:

Buenas noches, profesor, a continuación, le daré una breve explicación de como se hizo el código para que las cartas de YuGiOh se vieran, como ya le había explicado, es algo meramente estético, que no afecta a la lógica del juego y en lo que se utilizaron bibliotecas de HTTPS y URL para java:

¿Qué hace el código?

El objetivo de este código y clases es que se puedan mostrar las imágenes reales del juego de cartas de YuGiOh en la interfaz del juego, para esto, se consulta el sitio “YGOPRODeck”, una base de datos con todas las cartas de YuGiOh

Funcionalidad de cada archivo:

ImagenCartaCache.java - Descarga las imágenes y notifica cuando están listas

CartaVisual.java - Panel visual de cada carta en la mano: imagen

SlotCampoVisual.java - Panel visual de una carta en el campo: imagen, rotación en defensa y estadísticas

VentanaDuelo.java – Se usan todos los anteriores archivos en la función ActualizarInterfaz ()

ImagenCartaCache.java

Esta es la parte mas compleja de los archivos creados. Su trabajo es pedir la imagen de cada carta a internet, descargarla sin bloquear la pantalla, guardarla en memoria y avisar cuando está lista.

¿Qué es un Singleton?: Este es un patrón o modelo que garantiza que solo se descargue una imagen por objeto para que no se descargue la misma imagen varias veces

¿Qué es ConcurrentHashMap?: Es un diccionario que asocia cada carta con la imagen descargada en base a su nombre, lo que lo hace más útil que HashMap normal es que permite usar varios hilos a la vez y que no se corrompan los datos

¿Qué es SwingWorker?: Swing es la librería de interfaces de java, pero SwingWorker es una clase diseñada para ejecutar el código lento por separado y que se use el resultado cuando ya esté listo

¿Qué es un ImagenIcon?: Es una interfaz funcional que representa una acción que recibe un parámetro, pero no devuelve nada

Consulta de YGOPRODeck: Este es un sitio que guarda las cartas de YuGiOh en formato JSON lo cual es por así decirlo, el conjunto de datos de la imagen los cuales están guardados como un objeto de JavaScript, esto permite recrear la imagen en el código sin necesidad de descargar un archivo JPEG por decir alguno

Placeholder Dorado: Durante los segundos que tarda en descargarse la imagen se muestra un espacio con bordes dorados en donde se cargara la carta en un par de segundos

CartaVisual.java

Extiende JPanel y dibuja las cartas visualmente desde la imagen que le entrega el archivo anteriormente explicado, en caso de que esta no se encuentre en la base de datos, aparecerá el placeholder

PaintComponent: En lugar de usar JLabels y demás se usa el PaintComponent que java usa para redibujar el aspecto de la imagen

Hover: Este es un efecto que aparece cuando el puntero se para sobre la carta para recalcar sobre que carta se encuentra

SlotCampoVisual.java

Este es lo mismo que la CartaVisual solo que este se aparece únicamente en el campo, o sea, sobre las cartas ya usadas

posición: Se usa la capacidad de transformación de Graphics2D, una clase de java para rotar la imagen y que se vea que está en posición defensiva u ofensiva

Overlay de estadísticas: Sobre la imagen del monstruo se puede ver el ataque o defensa del monstruo dependiendo de la posición que se haya usado

Aclaraciones por si las dudas:

¿Qué es HTTPS y HttpRequest?: El primero es una clase de java que permite hacer peticiones HTTP y el segundo es una petición que se hace antes de enviarse. Ejemplo:

```
private String consultarUrlImagen(String nombre) {
    try {
        String encoded = java.net.URLEncoder.encode(nombre, enc: "UTF-8");
        HttpRequest req = HttpRequest.newBuilder()
            .uri(URI.create("https://db.ygoprodeck.com/api/v7/cardinfo.php?name=" + encoded))
            .header(name: "Accept", value: "application/json")
            .build();
```

¿Qué es Synchronized en GetInstance()?: Esto se hace porque el SwingWorker crea partes del programa que corren por separado, esto impide que se bloqueen estas instancias obligando a que estas partes entren una por una al GetInstance()

¿Por qué Graphics2D y no solo Graphics?: Esto es porque la primera es una subclase de la segunda la cual tiene más capacidades como la transformación y el control de renderizado

Referencias:

SwingWorker

Tutorial en español — ChuWiki:

https://chuidiang.org/index.php?title=Ejemplo_sencillo_con_SwingWorker

Tutorial práctico — GeeksforGeeks (inglés):

<https://www.geeksforgeeks.org/swingworker-in-java/>

Patrón Singleton

DigitalOcean — Java Singleton Design Pattern (inglés):

<https://www.digitalocean.com/community/tutorials/java-singleton-design-pattern-best-practices-examples>

Baeldung — Thread-Safe Singleton (inglés): <https://www.baeldung.com/java-implement-thread-safe-singleton>

ConcurrentHashMap

GeeksforGeeks — ConcurrentHashMap (inglés):

<https://www.geeksforgeeks.org/concurrenthashmap-in-java/>

Baeldung — Reading and Writing with ConcurrentHashMap (inglés):

<https://www.baeldung.com/concurrenthashmap-reading-and-writing>

HttpClient y peticiones HTTP

Javadoc oficial — HttpClient Java 11 (inglés):

<https://docs.oracle.com/en/java/javase/11/docs/api/java.net.http/java/net/http/HttpClient.html>

Javadoc oficial — Consumer (inglés):

<https://docs.oracle.com/javase/8/docs/api/java/util/function/Consumer.html>

GeeksforGeeks — Consumer Interface (inglés): <https://www.geeksforgeeks.org/java-8-consumer-interface-in-java-with-examples/>

Baeldung — Functional Interfaces (inglés): <https://www.baeldung.com/java-8-functional-interfaces>

paintComponent y Graphics2D

Tutorial completo — NTU Singapore (inglés):
https://www3.ntu.edu.sg/home/ehchua/programming/java/J8b_Game_2DGraphics.html

GeeksforGeeks — JComponent y paintComponent (inglés):
<https://www.geeksforgeeks.org/java-jcomponent/>

Baeldung — Drawing con Graphics2D (inglés): <https://www.baeldung.com/java-swing-draw-circle>